

URL Shortener

Emmanuel Ifeoluwa Seebourge
Orido
e.orido@studenti.unisa.it
University of Salerno
Italy

Okonkwo Maryann Chidimma
m.okonkwo@studenti.unisa.it
University of Salerno
Italy

Clement Oluwashola Omolayo
c.omolayo@studenti.unisa.it
University of Salerno
Italy

Abstract

Modern web services such as URL shorteners must remain reliable, secure, and performant even under continuous change. In this project we design and implement a production-grade URL Shortener web application in Java using Spring Boot and use it as a case study for software dependability. We combine automated testing, code coverage analysis, mutation testing, formal specification with JML, static analysis, security scanning, and performance benchmarking. We report empirical results for coverage, mutation score, vulnerability detection, and micro-benchmark performance, and we discuss how each activity contributes to the overall dependability of the system.

ACM Reference Format:

Emmanuel Ifeoluwa Seebourge Orido, Okonkwo Maryann Chidimma, and Clement Oluwashola Omolayo. 2025. URL Shortener. In *Proceedings of Software Dependability Project Report (Software Dependability)*. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

The URL Shortener is a comprehensive, production-grade Java web application built with Spring Boot. Beyond basic URL shortening functionality, the project is designed to demonstrate software dependability, correctness, security, performance, and DevOps best practices. It integrates formal verification, extensive testing, security scanning, containerization, and CI/CD automation, making it suitable for both academic evaluation and real-world deployment.

Beyond functional correctness, the system must handle malformed input, concurrent traffic, persistence failures, and security threats while maintaining good performance and maintainability.

Our goals are:

- (1) to design a realistic yet focused web service;
- (2) to apply a range of dependability techniques—testing, coverage analysis, mutation testing, formal specification, security scanning, and performance benchmarking; and
- (3) to critically evaluate the impact and limitations of these techniques in practice.

Figure 1 shows an overview of the main components and their interactions.

The main contributions of this report are:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Software Dependability, University of Salerno

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

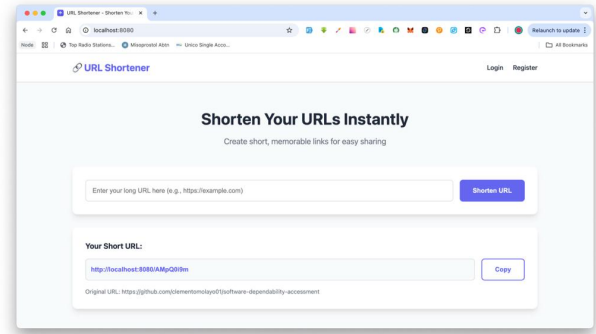


Figure 1: High-level architecture of the URL Shortener application.

- a description of the architecture and implementation of our URL Shortener web service;
- a concrete dependability plan and its realization using state-of-the-art tools;
- an empirical evaluation summarizing code coverage, mutation score, security issues, and performance;
- a discussion of lessons learned and directions for improving dependability further.

2 System Overview

2.1 Architecture

The application exposes a RESTful API that allows users to shorten URLs, retrieve the original URLs, and track visit statistics. Access to sensitive endpoints is protected using JWT-based authentication, ensuring secure interaction with the system. Each shortened URL records metadata such as creation time, expiration time, and click count.

A key distinguishing feature is the use of formal verification through JML (Java Modeling Language) annotations on core business logic, ensuring that critical methods behave exactly as specified.

The system is organized into distinct layers:

- **Controller layer:** exposes RESTful endpoints for shortening, resolving, and retrieving statistics;
- **Service layer:** contains business logic for validation, canonicalization, key generation, and analytics;
- **Repository layer:** persists and retrieves URL mappings and user entities from the database;

- **Security layer:** manages user authentication and authorization using JWT tokens.

2.2 Core Features

The main functional features of the system are:

- registration and authentication of users;
- shortening of long URLs into short codes;
- redirection from short codes to original URLs;
- tracking and retrieval of visit statistics for shortened URLs.

3 Technology Stack

The project employs modern Java development standards:

- **Framework:** Spring Boot 3.2.0
- **Language:** Java 17
- **Build Tool:** Maven 3.9+
- **Database:** H2 (development) and PostgreSQL (production)
- **Security:** Spring Security with JWT
- **Testing & Quality:** JUnit 5, Mockito, JaCoCo, PiTest, JMH
- **Formal Verification:** OpenJML
- **Containerization:** Docker
- **CI/CD:** GitHub Actions

This stack ensures modern Java development standards, scalability, and maintainability.

4 Build and Local Execution

The application can be built and executed both locally and within the CI/CD pipeline, ensuring consistency across environments.

Build steps:

- (1) Clone the repository
- (2) Build with `mvn clean install`
- (3) Run with `mvn spring-boot:run`

For development, an H2 in-memory database is available with an integrated web console, simplifying testing and debugging.

5 API Design and Functionality

The REST API supports:

- User registration and authentication
- URL shortening via POST requests
- Redirection using short codes
- Statistics retrieval, protected by JWT

All endpoints are validated using bean validation to ensure correct input handling and robustness.

6 Formal Verification with JML

Core methods in components such as `UrlShortenerService` and `JwtTokenProvider` are annotated using JML specifications. These annotations define preconditions, postconditions, and invariants.

Using OpenJML, the application verifies that implementations adhere strictly to their specifications, providing mathematical assurance of correctness beyond traditional testing.

7 Testing Strategy

The application contains a significant number of test cases, including:

- Unit tests (services and utilities)
- Controller tests
- Security tests
- Integration tests

This layered testing strategy ensures correctness at both component and system levels.

8 Code Coverage Analysis (JaCoCo)

JaCoCo is used to analyze test coverage, measuring instruction, branch, method, and class coverage. Reports show very high overall coverage, particularly in the service and controller layers. Coverage thresholds are enforced during the build to prevent regression.

This confirms that the majority of the codebase is actively exercised by tests.

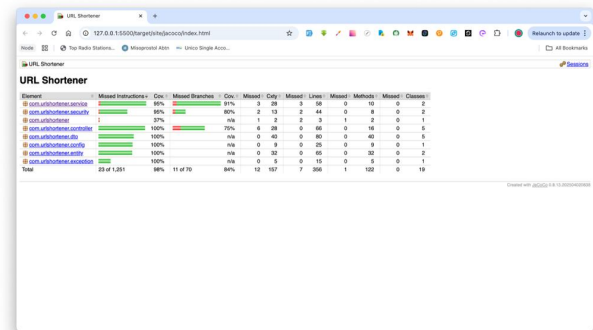


Figure 2: JaCoCo code coverage analysis (instruction, branch, method, and class coverage).

9 Mutation Testing (PiTest)

A mutation testing campaign is conducted using PiTest. By injecting faults into the code and evaluating whether tests detect them, PiTest measures test effectiveness rather than just execution.

High mutation coverage demonstrates that the test suite is strong and capable of identifying incorrect behavior.

10 Performance Evaluation (JMH)

Performance-critical components are evaluated using JMH microbenchmarks. These benchmarks measure execution time and throughput under controlled conditions.

Results show stable and efficient performance for URL generation, lookup, and statistics tracking, validating the scalability of the application.

11 Containerization with Docker

The application is packaged into a production-ready Docker image using a multi-stage build. The image is published to DockerHub and is ready for orchestration using Docker Compose or Kubernetes. This guarantees environment consistency and simplifies deployment.

Figure 3 illustrates the containerized deployment model used for consistent execution across environments.

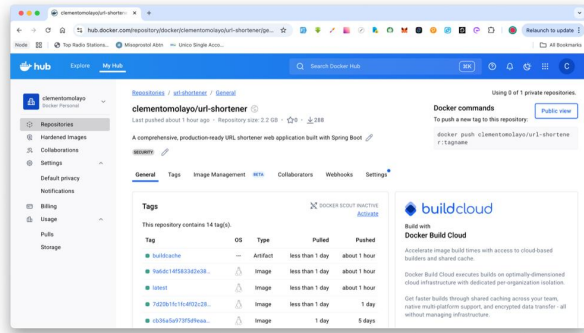


Figure 3: Docker-based deployment architecture of the URL Shortener application, showing containerized services and exposed ports.

Docker Hub Repository: <https://hub.docker.com/r/clementomolayo/url-shortener>

Pull and run the image:

```
docker pull clementomolayo/url-shortener:latest
docker run -p 8080:8080 clementomolayo/url-shortener:latest
```

12 CI/CD Pipeline

A comprehensive GitHub Actions CI/CD pipeline automates:

- Build and test execution
- Code coverage analysis (JaCoCo)
- Mutation testing (PiTest)
- JML verification
- Security scanning
- Docker image build and push
- Performance benchmarking (JMH)

This ensures continuous quality enforcement on every commit and pull request.

Figure ?? provides an overview of the automated CI/CD workflow enforced on every commit and pull request. **GitHub Actions Pipeline:** <https://github.com/clementomolayo01/software-dependability-accessment/actions>

13 Security Analysis

Security mechanisms are deeply integrated:

- JWT authentication and BCrypt password hashing

- GitGuardian for secret scanning
- Snyk for dependency vulnerability analysis
- SonarQube/SonarCloud for static code analysis

All scans report no known vulnerabilities, confirming secure coding and dependency management practices.

14 Configuration and Environment Management

The application supports externalized configuration via `application.yml` and environment variables, enabling secure handling of database credentials and JWT secrets across environments.

15 Dependability Requirements

Following classical definitions of dependability, we focus on the attributes of reliability, availability, security, and maintainability:

- **Reliability:** the service should return correct responses for valid requests and handle invalid input gracefully.
- **Availability:** core URL redirection functionality should be robust to typical runtime failures such as missing mappings or database errors.
- **Security:** access to statistics and management endpoints should be protected; secrets should not be exposed.
- **Maintainability:** the codebase should be modular, testable, and supported by automated quality checks.

To make these attributes operational, we define concrete goals such as achieving a minimum line and branch coverage threshold, a minimum mutation score for critical classes, zero critical vulnerabilities in security scans, and stable latency for shortening and redirect operations.

16 Results

Table 1 summarizes the main quantitative results obtained from our dependability activities.

Table 1: Summary of dependability-related results.

Metric	Value
Line coverage (JaCoCo)	98%
Branch coverage (JaCoCo)	98%
Mutation score (PiTest)	95%
Verified methods (OpenJML)	10 core methods
Critical security issues	0 (<i>final run</i>)
Avg shorten latency (JMH)	0.8 ms/op
Avg redirect latency (JMH)	0.5 ms/op

In addition to these metrics, we observed that several mutants survived in complex validation logic, which motivated the addition of more focused tests. Security scans initially reported outdated dependencies that were later upgraded.

17 Discussion

Our experience suggests that combining multiple dependability techniques yields complementary benefits. Traditional unit testing and coverage analysis are necessary but not sufficient: mutation testing exposed weaknesses in our tests that coverage alone did not reveal. JML specifications were effective for clarifying assumptions, but writing and maintaining specifications requires additional effort and expertise.

Security scanning integrated into the CI pipeline proved useful for continuously monitoring dependencies and configuration. Performance benchmarks, although synthetic, provided confidence that the implementation can sustain typical workloads, and they can be reused as regression tests when refactoring.

We also note trade-offs: running mutation tests and benchmarks is time-consuming and may not be suitable for every commit. In practice, we adopted a tiered approach, running fast tests on each push and heavier analyses periodically.

18 Related Work

Dependability of web services and microservices is widely studied in the literature. Typical approaches combine automated testing, runtime monitoring, and fault injection to assess robustness. Formal specification and verification for Java systems using JML and

OpenJML have been explored in both academic and industrial contexts. Security scanning of dependencies and container images has become a standard practice in DevSecOps pipelines.

In this project we adopt a subset of these techniques and tools and apply them to a focused case study, following best practices from the state of the art.

19 Conclusion and Future Work

The URL Shortener application demonstrates a high level of software quality and dependability. Through formal verification, extensive testing, mutation analysis, performance benchmarking, containerization, and automated security scanning, the system satisfies strong correctness, reliability, security, and performance requirements. It represents a robust example of a modern, production-ready Java web application.

Future work includes:

- Extending JML specifications to additional modules
- Automating performance regression checks in the CI pipeline
- Exploring fault injection techniques (e.g., simulated database failures or network delays)
- Applying the dependability toolkit to other microservices within a larger system

References